Predicting stock prices with LSTM Networks

David Duemig

Abstract

We ask whether a long short term memory (LSTM) network can predict the direction of the S&P 500 over the next month from past returns, volatility and trading volume, and whether it does so any better than a regularised logistic regression. We use one non-overlapping observation per month, build stationary features, and validate with purged walk-forward cross-validation so that no future information leaks into the past. On a held-out test set of 106 months the LSTM, the linear baseline and a gradient-boosting model all score close to the naive rule of always predicting "up". Test AUCs lie between 0.44 and 0.62, and no model beats the base rate by a margin that survives a block-bootstrap confidence interval. A probability threshold trained only on validation data does not produce a timing strategy that beats buying and holding after costs. Monthly index direction remains very hard to predict from price and volume alone.

1 Introduction

Artificial intelligence now performs many tasks that once needed an expert. In finance this has encouraged a search for models that forecast asset returns. Models of individual stock returns usually start from company and market characteristics, and most studies form their predictions with simple linear models [1, 2]. A growing literature argues that machine learning and deep learning, which allow nonlinear interactions between predictors, can improve these forecasts [3, 4, 5].

This project asks a narrow version of that question. We train LSTM networks on time-series price and volume data for the S&P 500 and compare their out-of-sample predictability with that of a regularised logistic regression and a gradient-boosting classifier. The task is a monthly up or down call on the index. The headline result is negative, and we report it as it comes out: the problem is close to unpredictable at this horizon from these inputs.

2 Related work

Successful financial models are usually proprietary, so published work with state-of-the-art performance on this exact task is scarce. The academic community has nonetheless taken up deep learning in finance, and the LSTM is a common choice for sequence data. Gao applied an LSTM to forecast six US industries from 359 features [6]. Nelson et al predicted short-horizon direction from technical features [8], and Chen et al used an LSTM on historical prices to classify daily moves [7]. Lopez de Prado documents why ordinary cross-validation leaks in finance and proposes purging and an embargo, which we adopt [9]. Our features are standard momentum, volatility and liquidity signals.

3 Dataset and Features

3.1 Raw Data

The data are daily quotes for the S&P 500 price index (Yahoo ticker `^GSPC`) from January 1980 to December 2025, downloaded with `yfinance`. Each row has the open, high, low, close and volume. We use the close and the volume. The raw series has about 11,595 trading days. Re-downloading later may give marginally different values, which does not change the conclusions.

3.2 Features

All features are stationary, which matters because the model must generalise across four decades in which the index level and the traded volume both grew by large factors. From the daily close we form log returns,

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right). \tag{1}$$

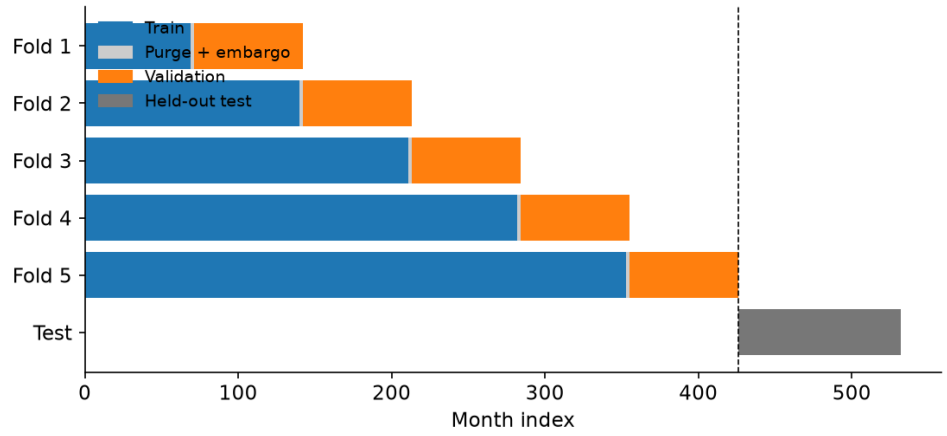


Figure 1: Purged walk-forward cross-validation on the development months. Each fold trains on an expanding block of past months, skips a purge and embargo gap, and validates on the next block. The final months are held out as the test set.

For each look-back window of w weeks (five trading days per week), with w in $\{1, 2, 3, 4, 8, 12, \ldots, 80\}$, we compute three month-end features: the summed log return over the window, the standard deviation of daily returns over the window, and a detrended log volume. The first two are stationary by construction. For volume we subtract a trailing one-year mean of log volume from the window mean of log volume,

$$\text{volume}_{w,t} = \overline{\ln V}_{t,5w} - \overline{\ln V}_{t,252}, \tag{2}$$

which removes the strong secular growth in turnover. This gives 23 windows for each of the three families and 69 month-end features in total. The three families, volatility, return and volume, are combined into the seven models of the original project (each family alone, each pair, and all three).

The LSTM does not see these aggregates. It receives the most recent 60 trading days of the daily channels that correspond to the chosen families: the daily log return, a rolling daily volatility, and the detrended daily log volume. This lets the network read a real sequence rather than a single snapshot.

3.3 Scaling

Features are rescaled before training so that the optimiser sees comparable magnitudes and the penalty treats coefficients evenly. We use standardisation, which centres each feature and scales it to unit variance,

$$X_z^{(i)} = \frac{X^{(i)} - \bar{X}}{s_X}. \tag{3}$$

The scaler is fitted inside each cross-validation fold on the training part only, never on the validation or test months, so no scaling information leaks forward. The LSTM channels are standardised the same way, using statistics from the training sequences only.

3.4 Separating Test Data

Ordinary k-fold cross-validation fails in finance because it mixes future and past [9]. We instead hold out the last 20% of the months as a test set, used once after everything is fixed, and tune on the earlier months with walk-forward cross-validation. Because each label looks one month ahead, a training month whose label window reaches a validation month is purged, and a further embargo of 1 month is dropped before each validation block. Figure 1 shows the scheme.

3.5 Label Distribution

Each month is labelled up or down by the sign of the next month's return. Using one non-overlapping observation per month removes the heavy overlap of the original daily labelling, at the cost of a small sample: 532 months after the feature warm-up, of which 426 are development months and 106 are

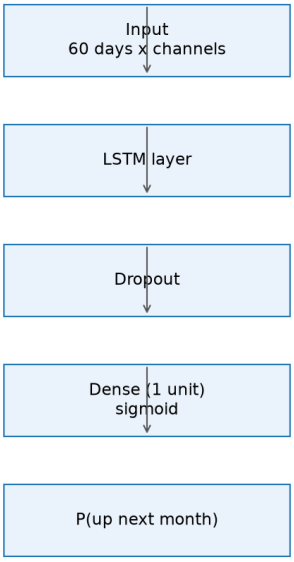


Figure 2: Network structure. A sequence of daily feature channels feeds one LSTM layer, a dropout layer and a single sigmoid output.

test months. The classes are unbalanced in favour of "up": 62.0% of development months and 68.9% of test months are up. This imbalance is central to reading the results, because a model that always predicts "up" already scores 68.9% on the test set.

4 Models

4.1 Baseline Models

Two baselines use the month-end aggregates. The first is a logistic regression with an elastic-net penalty, which spans ridge and lasso, trained with the saga solver. The penalty strength and the L1 ratio are tuned by the walk-forward cross-validation above. The second is a gradient-boosting classifier, tuned over its learning rate and tree size on the same folds. We also report the naive always-up rule, which is the benchmark every model has to beat.

4.2 LSTM Networks

The LSTM reads a sequence of 60 trading days and outputs the probability of an up month through a single sigmoid unit (Figure 2). An LSTM keeps a cell state across time steps and uses forget, input and output gates to decide what to keep, add and read out, which lets it learn longer dependencies than a plain recurrent network [10]. Because the sample is small, the architecture is kept deliberately small and is not tuned; instead we train it with early stopping and average over 10 random seeds, reporting the mean and the spread.

4.3 Experiment Design

For each of the seven feature sets we fit the elastic-net baseline, the gradient-boosting baseline and the LSTM. We score each model on the test months by accuracy, by the area under the ROC curve (AUC) and by the Brier score, and we attach 95% confidence intervals from a moving-block bootstrap over months, which preserves any serial dependence. We also compare each model with the always-up rule month by month and report whether the difference is positive after the bootstrap. Finally we test a simple timing strategy: hold the index next month when the predicted probability of an up month exceeds a threshold chosen on validation data only, otherwise hold cash, and charge a transaction cost of 10 basis points on each change in position.

Table 1: Test-set accuracy and AUC for each model and feature set, with the always-up benchmark. EN is the elastic-net logistic regression and GB the gradient-boosting model. The test set has 106 months with an up rate of 68.9%.

		Test accuracy			Test AUC		
Set	Features	EN	GB	LSTM	EN	GB	LSTM
1	Volatility	0.69	0.60	0.68	0.48	0.47	0.49
2	Return	0.66	0.56	0.69	0.52	0.47	0.54
3	Volume	0.56	0.58	0.69	0.51	0.58	0.45
4	Volatility + Return	0.68	0.58	0.69	0.55	0.48	0.53
5	Volatility + Volume	0.69	0.61	0.69	0.51	0.55	0.44
6	Return + Volume	0.64	0.65	0.69	0.51	0.58	0.48
7	Volatility + Return + Volume	0.69	0.59	0.69	0.62	0.53	0.53
Always up			0.69			0.50	

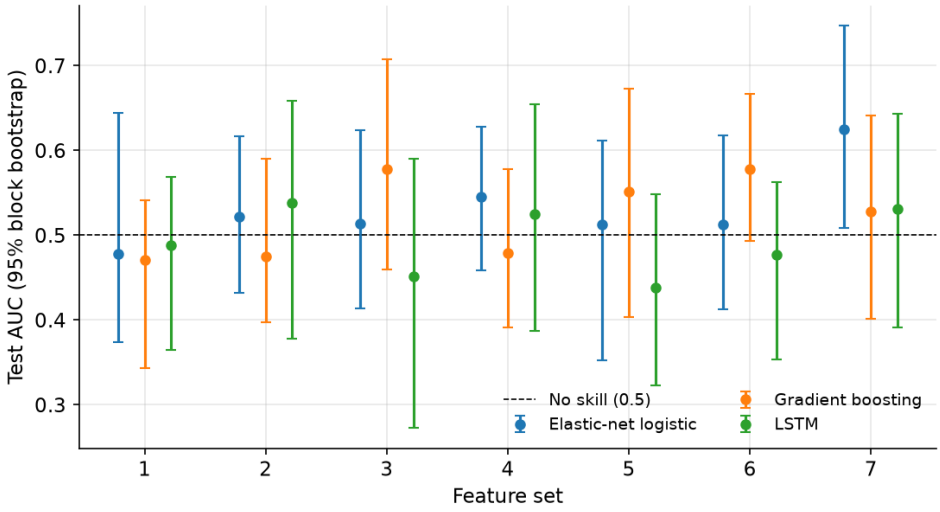


Figure 3: Test AUC by feature set and model, with 95% block-bootstrap intervals. The dashed line marks 0.5, the no-skill level.

4.4 Experiment Results

Table 1 summarises the classification results and Figure 3 plots the test AUCs with their intervals. The picture is consistent across models and feature sets. Test AUCs cluster around 0.5, from 0.44 to 0.62, and all but one of the intervals contain 0.5. The best test accuracy is 68.9%, equal to the always-up rate of 68.9%; no model beats always-up by an accuracy margin whose confidence interval excludes zero. One of the twenty-one model and feature-set combinations, the elastic net on all three families, has an AUC interval that just clears 0.5. With twenty-one combinations this is close to what chance alone produces, and that same model's accuracy does not beat always-up.

The economic test tells the same story. Table 2 and Figure 4 compare buying and holding with the best timing strategy per model family. Buy-and-hold earns an annual return of 12.8% at a Sharpe ratio of 0.85 over the test period. The best timing strategy reaches a Sharpe of 1.00, but it is the best of twenty-one model and feature-set combinations, and most strategies simply stay invested and recover buy-and-hold. An edge of this size is well within what picking the best of many would produce, so it is not evidence of genuine timing skill.

Table 2: Annualised test performance, buy-and-hold against the best timing strategy per model family, after costs.

Strategy	Ann. return	Ann. vol	Sharpe	Max drawdown
Buy and hold	12.8%	15.7%	0.85	-24.8%
Elastic net (set 1)	12.8%	15.7%	0.85	-24.8%
Gradient boosting (set 3)	14.3%	14.7%	0.99	-17.7%
LSTM (set 3)	14.7%	14.9%	1.00	-22.5%

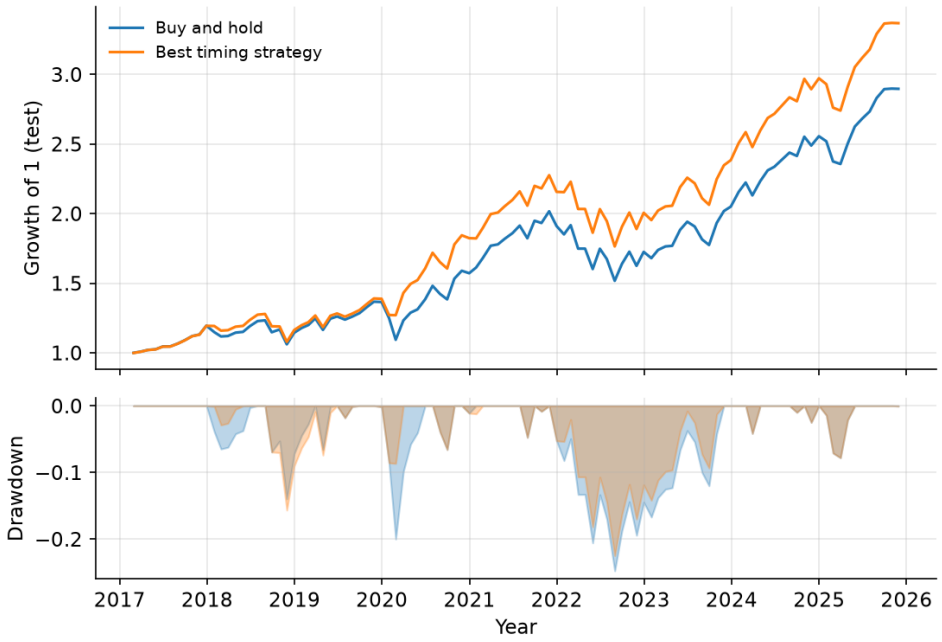


Figure 4: Growth of one unit over the test period for buy-and-hold and the best timing strategy, with drawdowns below.

5 Conclusion and Future Work

Monthly index direction remains very hard to predict even after cleaning up the method. With non-overlapping labels, purged validation, stationary features and a sequence model that sees real history, the LSTM, the linear baseline and the gradient-boosting model all sit close to the always-up rule, and the timing strategy does not beat buying and holding after costs. One honest reading is that price and volume do not carry enough information about the next month's direction, and that the strong upward drift of the index makes the base rate hard to beat. Useful next steps are to add information beyond price and volume, such as fundamentals or macro data; to predict a continuous return or a quantile rather than a sign; and to study a longer or international sample so that the test set is not dominated by a single bull market.

Acknowledgements

This project began as a course project for CS230 Deep Learning. I thank the teaching team and my advisor Sagar Honnungar.

References

[1] Blitz, D. & Vidojevic, M. (2018), 'The Characteristics of Factor Investing', SSRN Electronic Journal, `https://ssrn.com/abstract=3206798`.

[2] Lewellen, J. (2015), 'The Cross-section of Expected Stock Returns', Critical Finance Review, vol. 4, no. 1, pp. 1–44, https://ssrn.com/abstract=2511246.

[3] Freyberger, J., Neuhierl, A. & Weber, M. (2017), 'Dissecting characteristics nonparametrically', CESifo Working Paper 6391, https://ssrn.com/abstract=2935432.

[4] Gu, S., Kelly, B.T. & Xiu, D. (2018), 'Empirical Asset Pricing via Machine Learning', Chicago Booth Research Paper 18-04, https://ssrn.com/abstract=3159577.

[5] Rapach, D., Strauss, J., Tu, J. & Zhou, G. (2018), 'Industry Return Predictability: A Machine Learning Approach', SSRN Electronic Journal, https://ssrn.com/abstract=3120110.

[6] Gao, Q. (2016), 'Stock market forecasting using recurrent neural network', University of Missouri, Columbia.

[7] Chen, K., Zhou, Y. & Dai, F. (2015), 'A LSTM-based method for stock returns prediction', Proceedings of the 2015 IEEE International Conference on Big Data, pp. 2823–2824.

[8] Nelson, D.M., Pereira, A.C. & de Oliveira, R.A. (2017), 'Stock markets movement prediction with LSTM neural networks', Proceedings of the 2017 International Joint Conference on Neural Networks, pp. 1419–1426.

[9] Lopez de Prado, M. (2018), 'Advances in Financial Machine Learning', Wiley, New Jersey.

[10] Olah, C. (2015), 'Understanding LSTM Networks', http://colah.github.io/posts/2015-08-Understanding-LSTMs/.

Github Repository

https://github.com/duemig/Stanford-Project-Predicting-stock-prices-using-a-LSTM-Network